AVL Tree Implementation

AVL Trees: Implementation

```
template <typename ElementType>
struct AvlNode {
    ElementType Element;
    AvlNode*     Left;
    AvlNode*     Right;
    int         Height;
};
typedef struct AvlNode *Position;
typedef struct AvlNode *AvlTree;
AvlTree MakeEmpty( AvlTree T );
Position Find( ElementType X, AvlTree T );
Position FindMin( AvlTree T );
Position FindMax( AvlTree T );
AvlTree Insert( ElementType X, AvlTree T );
AvlTree Delete( ElementType X, AvlTree T );
ElementType Retrieve( Position P );
```

AVL Trees: Implementation

```
int Height(Position P )
{
    if( P == NULL )
        return -1;
    else
        return P->Height;
}
```

AVL Trees: Insert Function

```
AvlTree Insert( ElementType X, AvlTree T ) {
   if ( T == NULL ) { /* Create and return a one-node tree */
      T = new AvlNode;
      T->Element = X;
      T->Left = T->Right = NULL;
   }
   else if( X < T->Element ) {
      T->Left = Insert( X, T->Left );
      if( Height( T->Left ) - Height( T->Right ) == 2 )
         if( X < T->Left->Element )
            T = SingleRotateWithLeft( T ); // RR rotation
         else
            T = DoubleRotateWithLeft( T );  // RL rotation
   }
   else if( X > T->Element ) {
      T->Right = Insert( X, T->Right );
      if( Height( T->Right ) - Height( T->Left ) == 2 )
         if( X > T->Right->Element )
            T = SingleRotateWithRight( T );  // LL rotation
         else
            T = DoubleRotateWithRight( T );  // LR rotation
   }  /* Else X is in the tree already; we'll do nothing */
   T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
   return T;
}
```

AVL Trees: Insert Function

```
AvlTree Insert( ElementType X, AvlTree T ) {
   if ( T == NULL ) { /* Create and return a one-node tree */
      T = new AvlNode;
      T->Element = X;
      T->Left = T->Right = NULL;
   }
   else if( X < T->Element ) {
      T->Left = Insert( X, T->Left );
      if( Height( T->Left ) - Height( T->Right ) == 2 )
         if( X < T->Left->Element )
            T = SingleRotateWithLeft( T ); // RR rotation
```


```
if ( T == NULL ) { /* Create and return a one-node tree */
   T = new AvlNode;
   T->Element = X;
   T->Left = T->Right = NULL;
}
```


```
            T = SingleRotateWithRight( T );  // LL rotation
         else
            T = DoubleRotateWithRight( T );  // LR rotation
   }  /* Else X is in the tree already; we'll do nothing */
   T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
   return T;
}
```

AVL Trees: Insert Function

```
AvlTree Insert( ElementType X, AvlTree T ) {
   if ( T == NULL ) { /* Create and return a one-node tree */
      T = new AvlNode;
      T->Element = X;
      T->Left = T->Right = NULL;
   }
   else if( X < T->Element ) {
      T->Left = Insert( X, T->Left );
      if( Height( T->Left ) - Height( T->Right ) == 2 )
         if( X < T->Left->Element )
            T = SingleRotateWithLeft( T ); // RR rotation
         else
            T = DoubleRotateWithLeft( T );  // RL rotation
   }
   else if( X < T->Element ) {
      T->Left = Insert( X, T->Left );
      if( Height( T->Left ) - Height( T->Right ) == 2 )
         if( X < T->Left->Element )
            T = SingleRotateWithLeft( T );  // RR rotation
         else
            T = DoubleRotateWithLeft( T );  // RL rotation
   }
}
```

AVL Trees: Insert Function

```
AvlTree Insert( ElementType X, AvlTree T ) {
    else if( X > T->Element ) {
        T->Right = Insert( X, T->Right );
        if( Height( T->Right ) - Height( T->Left ) == 2 )
            if( X > T->Right->Element )
                T = SingleRotateWithRight( T );  // LL rotation
            else
                T = DoubleRotateWithRight( T );  // LR rotation
    }
    /* Else X is in the tree already; we'll do nothing */
    }
    else if( X > T->Element ) {
        T->Right = Insert( X, T->Right );
        if( Height( T->Right ) - Height( T->Left ) == 2 )
            if( X > T->Right->Element )
                T = SingleRotateWithRight( T ); // LL rotation
            else
                T = DoubleRotateWithRight( T );  // LR rotation
    }  /* Else X is in the tree already; we'll do nothing */
    T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
    return T;
}
```

AVL Trees: Insert Function

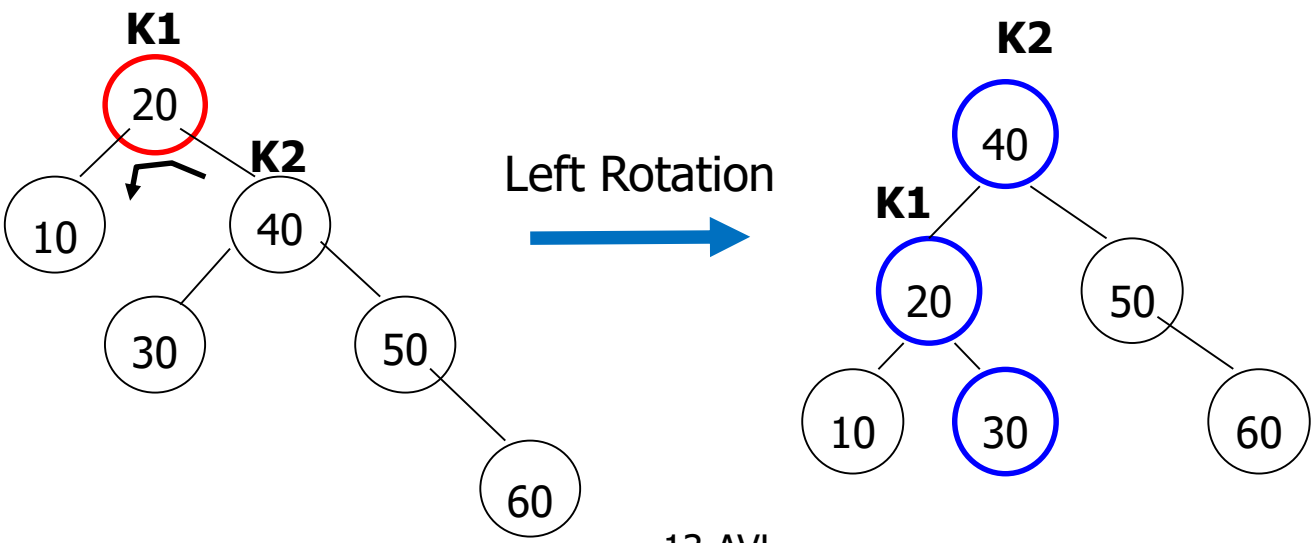
```
AvlTree Insert( ElementType X, AvlTree T ) {
   if ( T == NULL ) { /* Create and return a one-node tree */
      T = new AvlNode;
      T->Element = X;
      T->Left = T->Right = NULL;
   }
   else if( X < T->Element ) {
      T->Left = Insert( X, T->Left );
      if( Height( T->Left ) - Height( T->Right ) == 2 )
         if( X < T->Left->Element )
            T = SingleRotateWithLeft( T ); // RR rotation
         else
            T = DoubleRotateWithLeft( T ); // RL rotation
   }
   else if( X > T->Element ) {
      T->Right = Insert( X, T->Right );
      if( Height( T->Right ) - Height( T->Left ) == 2 )
```

```
T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
return T;
```

```
            T = DoubleRotateWithRight( T ); // LR rotation
   }  /* Else X is in the tree already; we'll do nothing */
   T->Height = Max( Height( T->Left ), Height( T->Right ) ) + 1;
   return T;
}
```

AVL Trees: LL Rotation

```
Position SingleRotateWithRight( Position K1 ) {
    Position K2;
    K2 = K1->Right;  // K1: node whose balance factor is violated
    K1->Right = K2->Left;
    K2->Left = K1;
    K1->Height = Max( Height(K1->Left), Height(K1->Right) ) + 1;
    K2->Height = Max( Height(K2->Right), K1->Height ) + 1;
    return K2;  /* New root */
}
```



AVL Trees: RR Rotation

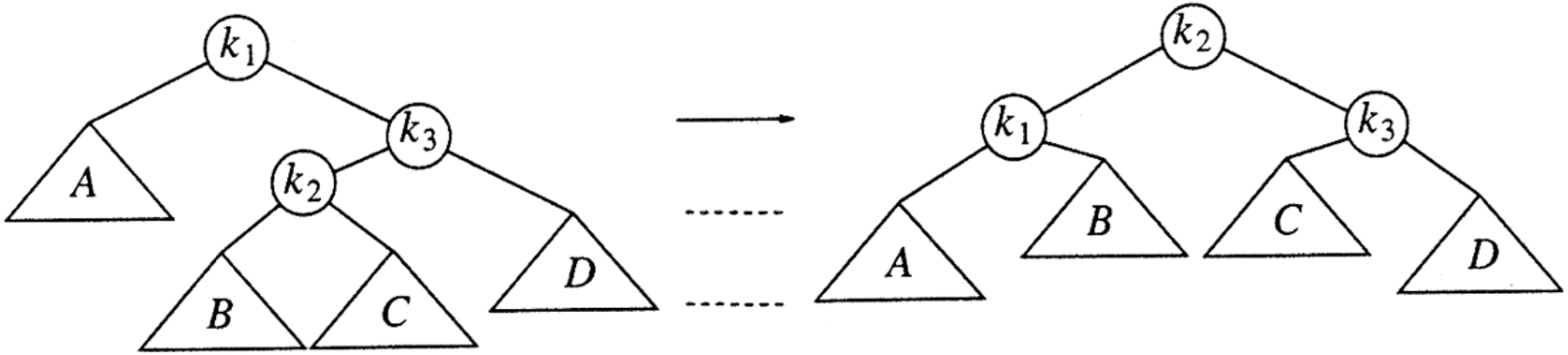
```
Position SingleRotateWithLeft( Position K1 ) {
   Position K2;
   K2 = K1->Left; // K1: node whose balance factor is violated
   K1->Left = K2->Right;
   K2->Right = K1;
   K1->Height = Max( Height(K1->Left), Height(K1->Right) ) + 1;
   K2->Height = Max( Height(K2->Left), K1->Height ) + 1;
   return K2;  /* New root */
}
```

AVL Trees: RL Rotation

```
Position DoubleRotateWithRight( Position K1)
{
    /* RR rotation between K3 and K2 */
    K1->Right = SingleRotateWithLeft( K1->Right );
    /* LL rotation between K1 and K2 */
    return SingleRotateWithRight( K1 );
}
```

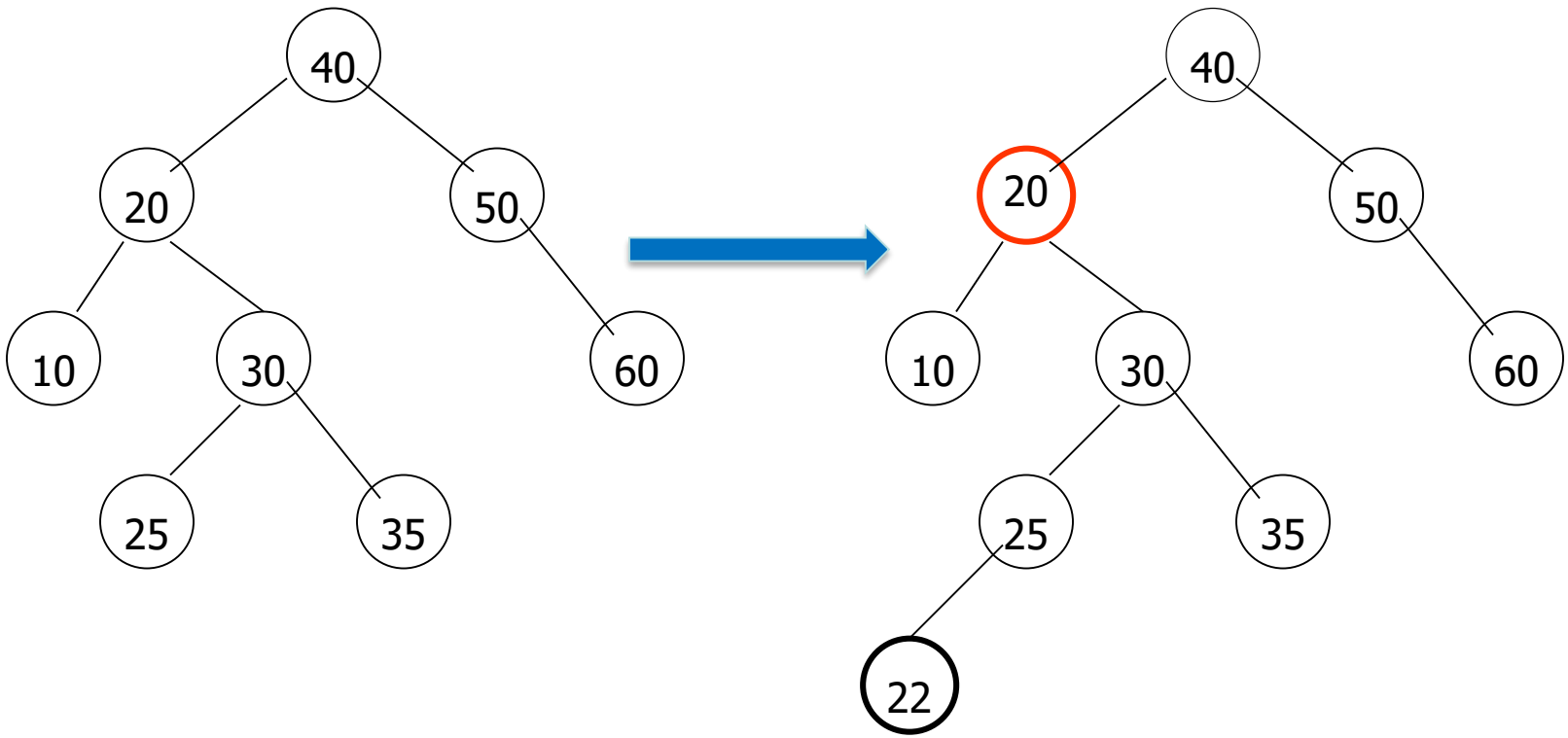
Single Right Rotation at K3

Single Left rotation at K1

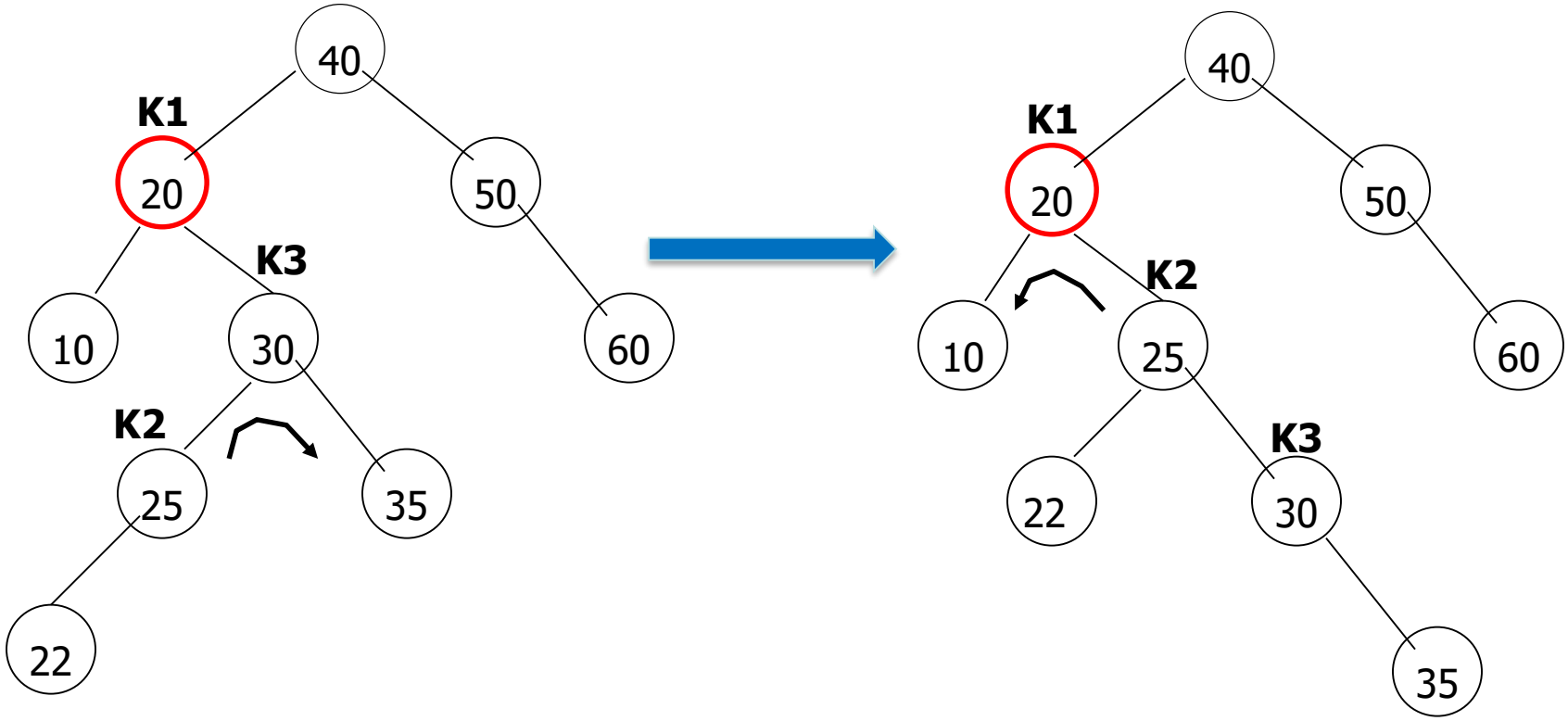


RL Rotation

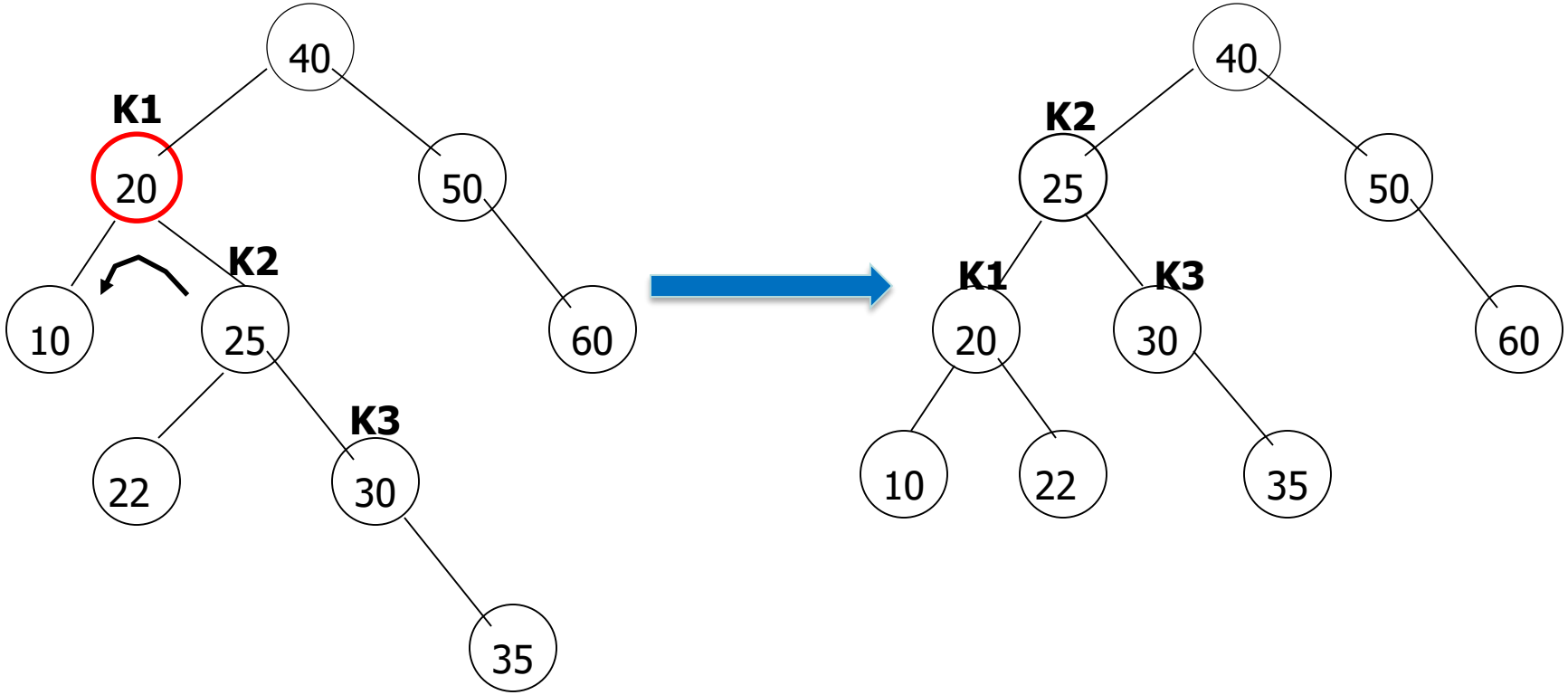
- Adding node 22
  - Requires double rotation!



RL Rotation



RL Rotation

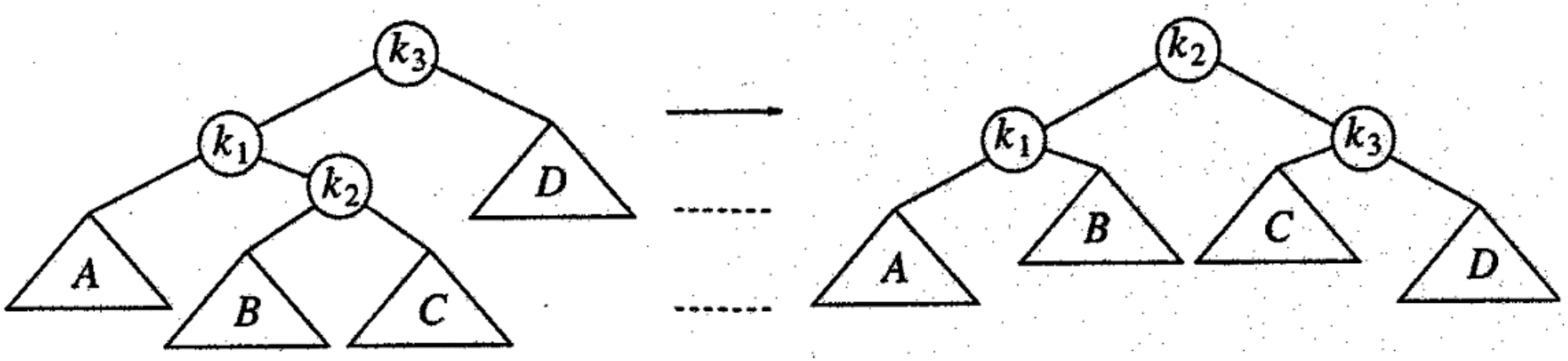


AVL Trees: LR Rotation

```
Position DoubleRotateWithLeft( Position K3 )
{
     /* LL rotation between K1 and K2 */
     K3->Left = SingleRotateWithRight( K3->Left );
     /* RR rotation between K3 and K2 */
     return SingleRotateWithLeft( K3 );
}
```

Single Left Rotation at K1

Single Right rotation at K3



AVL Tree Deletion

AVL Tree: Deletion

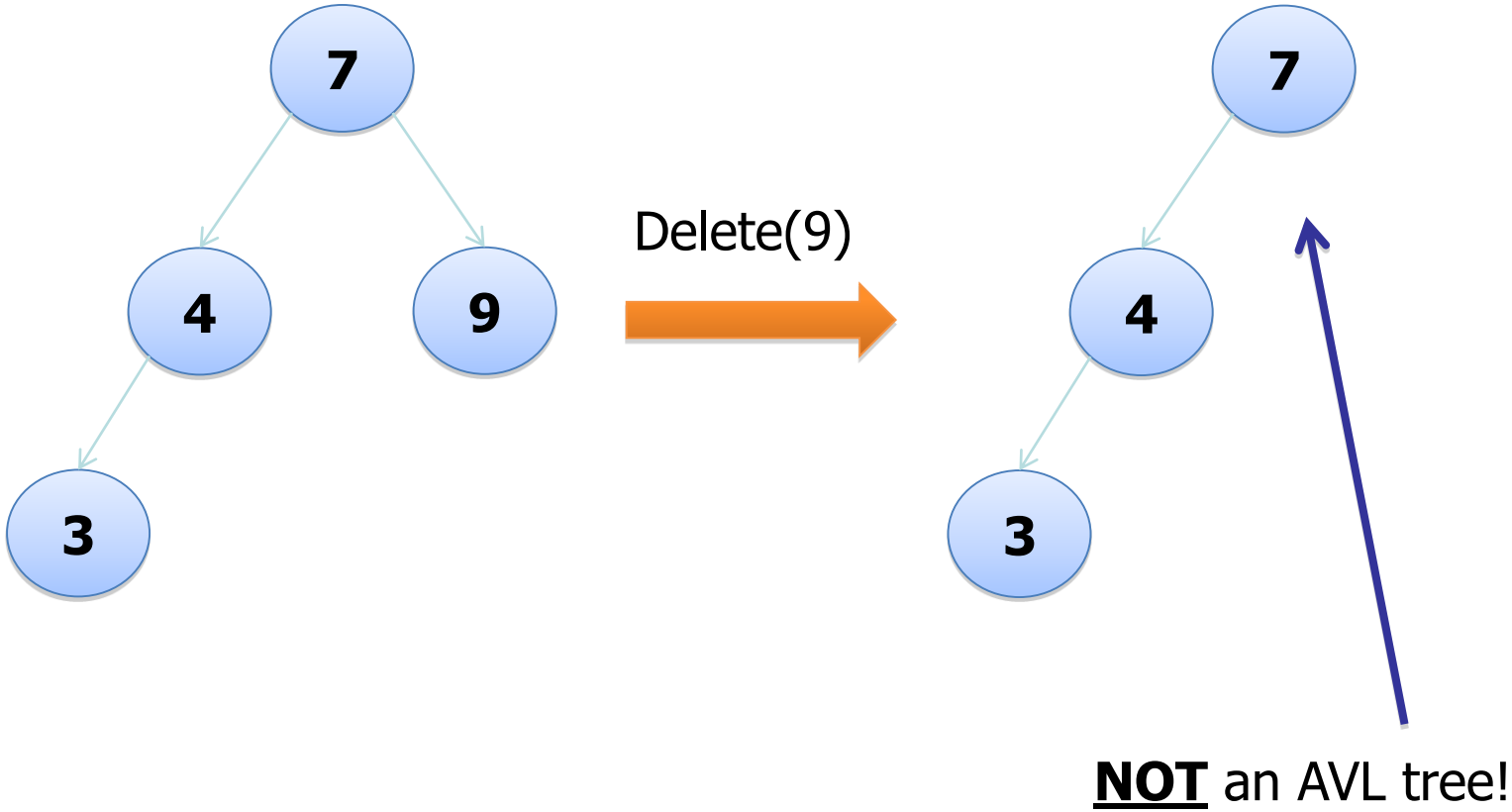
- Goal: To preserve the height balance property of BST after deletion

- Step 1: Perform BST delete
  - Maintains the BST property
  - May break the balance factors of ancestors!

- Step 2: Fix the AVL tree balance constraint
  - Perform transformation on the tree by means of rotation such that
    - BST property is maintained
    - Transformation fixes any balance factors that are < -1 or > 1

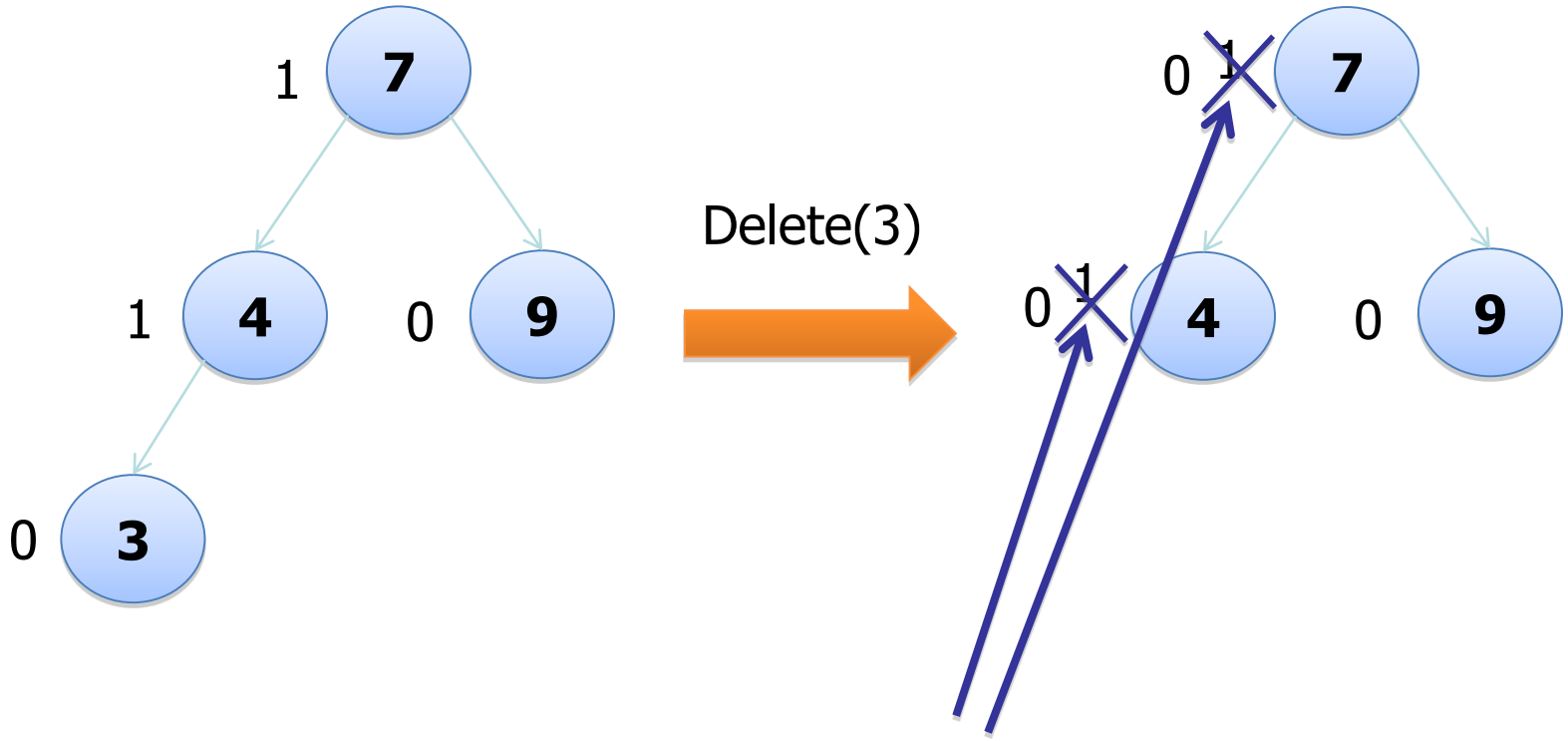
AVL Tree: Deletion

- BST deletion breaks the invariants of AVL tree



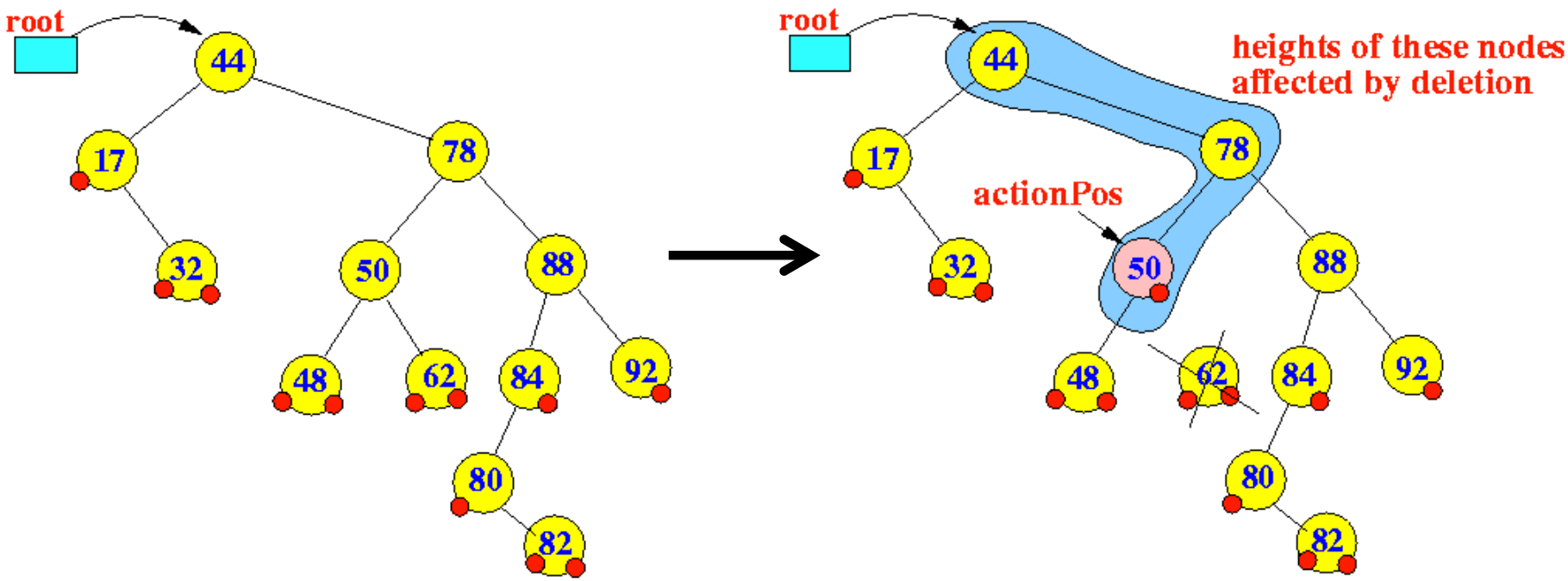
AVL Tree: Deletion

- BST deletion breaks the balance factors of ancestors



AVL Tree: BST Deletion

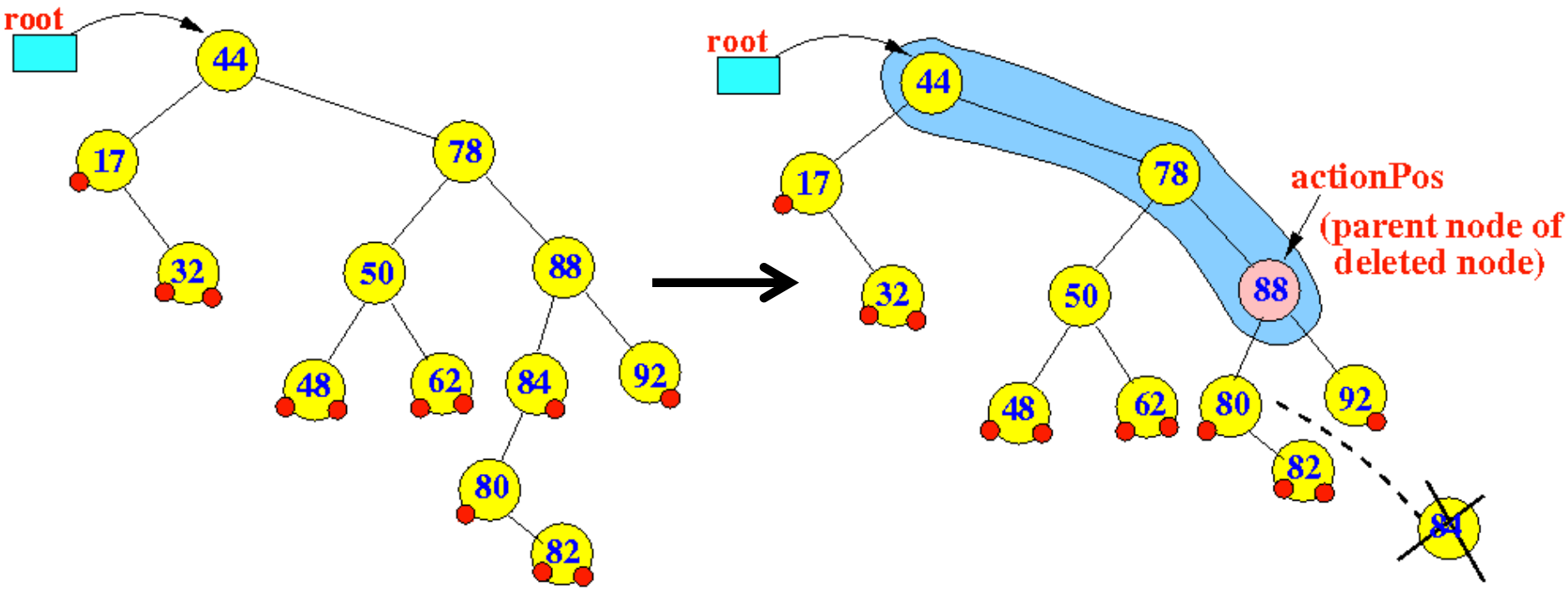
- Case 1: Node to be deleted has degree 0 (i.e., leaf node)
  - Consider deleting node containing 62



- Action position: Reference to parent node from which a node has been physically removed
  - First node whose height may be changed by deletion

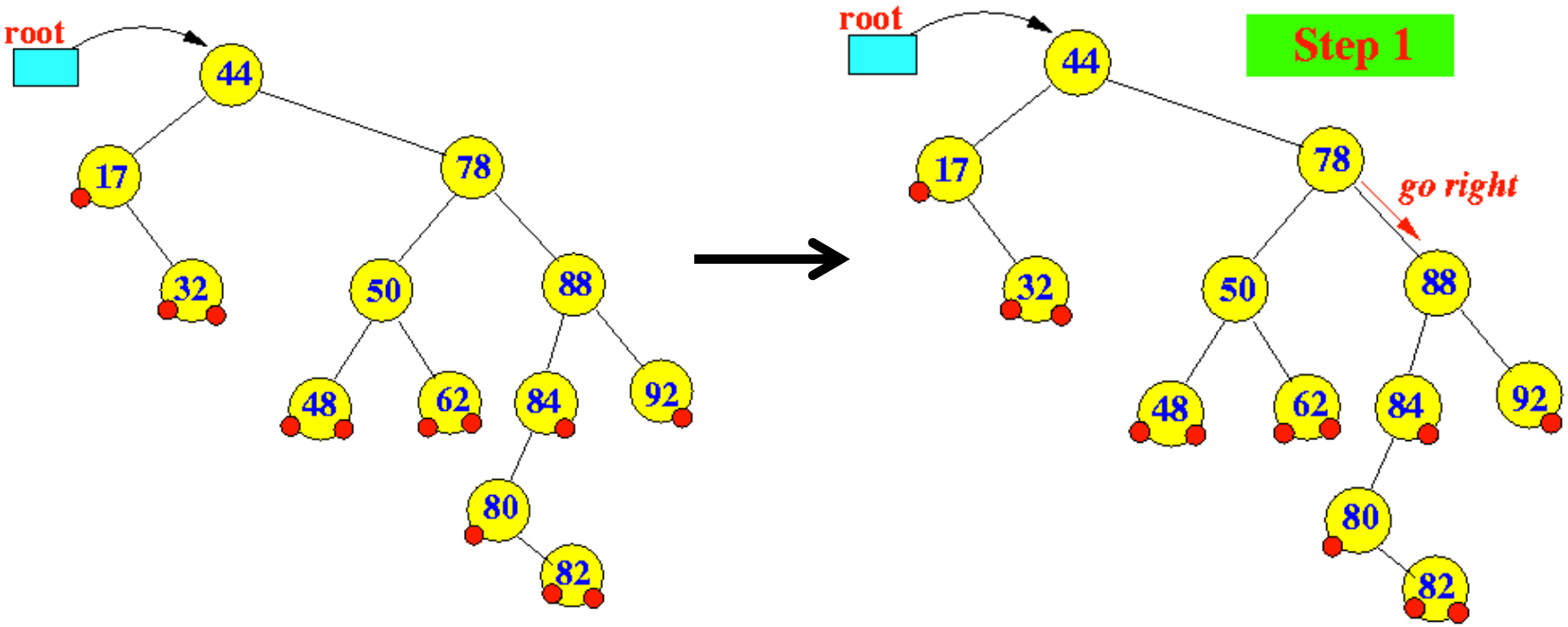
AVL Tree: BST Deletion

- Case 2: Node to be deleted has degree 1 (i.e., node with one child)
  - Consider deleting node containing 84



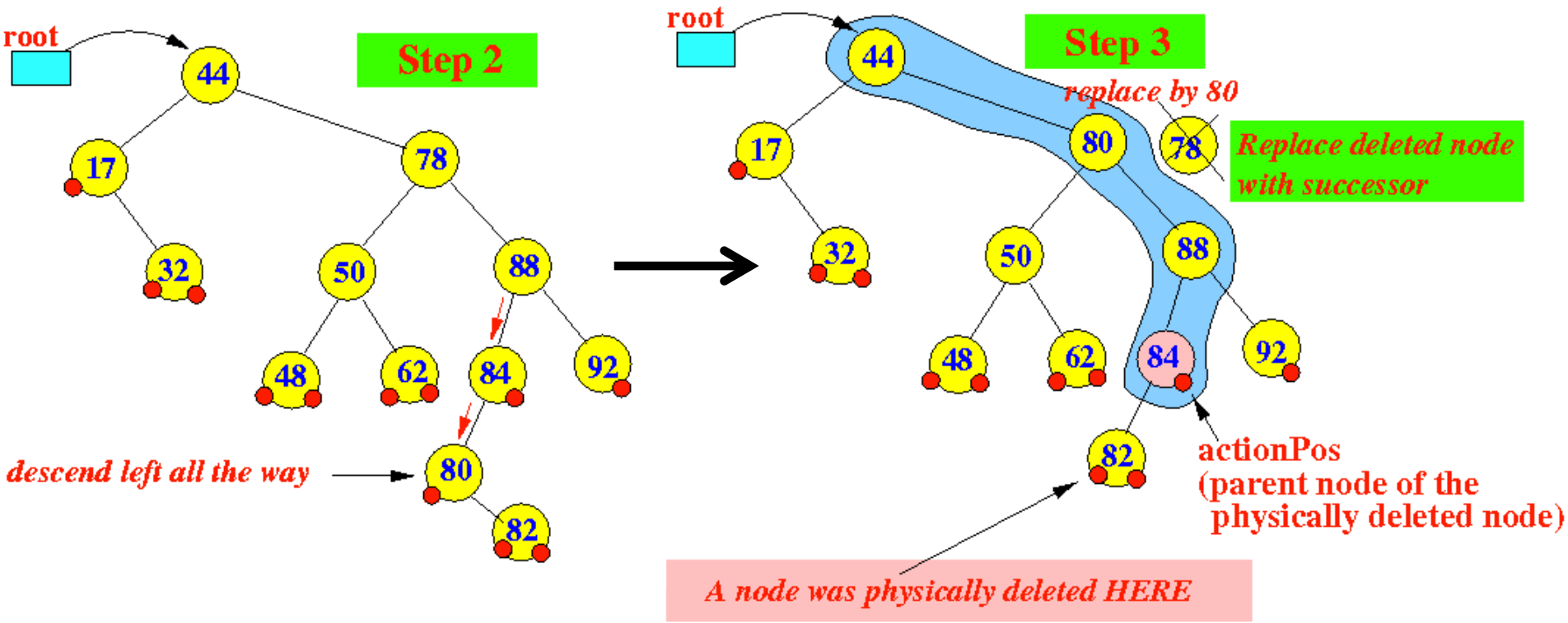
AVL Tree: BST Deletion

- Case 3: Node p to be deleted has two children (i.e., degree 2)
  - Replace node p with the minimum object in the right subtree
  - Delete that object from the right subtree
  - Consider deleting node containing 78



AVL Tree: BST Deletion

- Case 3: Node p to be deleted has two children (i.e., degree 2)
  - Replace node p with the minimum object in the right subtree
  - Delete that object from the right subtree
  - Consider deleting node containing 78



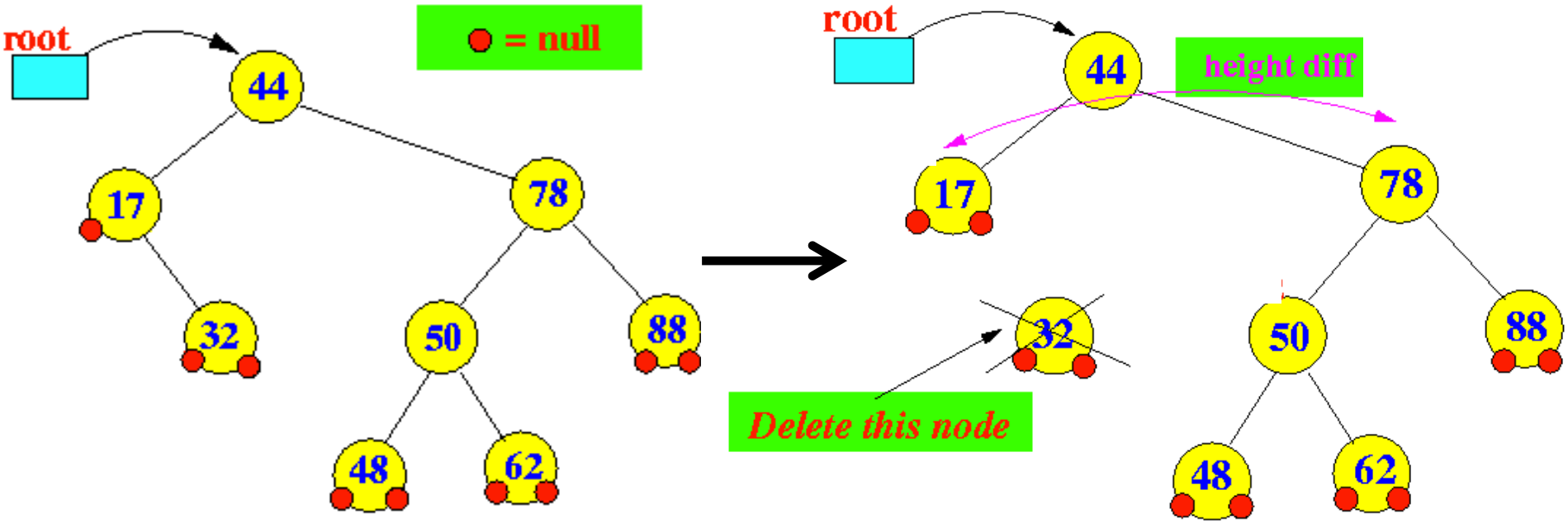
AVL Tree Deletion

- After removing a child, delete must check for imbalance
  - Similar to insert operation

- Rotations can be used to re-balance an out-of-balanced AVL tree
  - LL, RR, LR and RL rotations

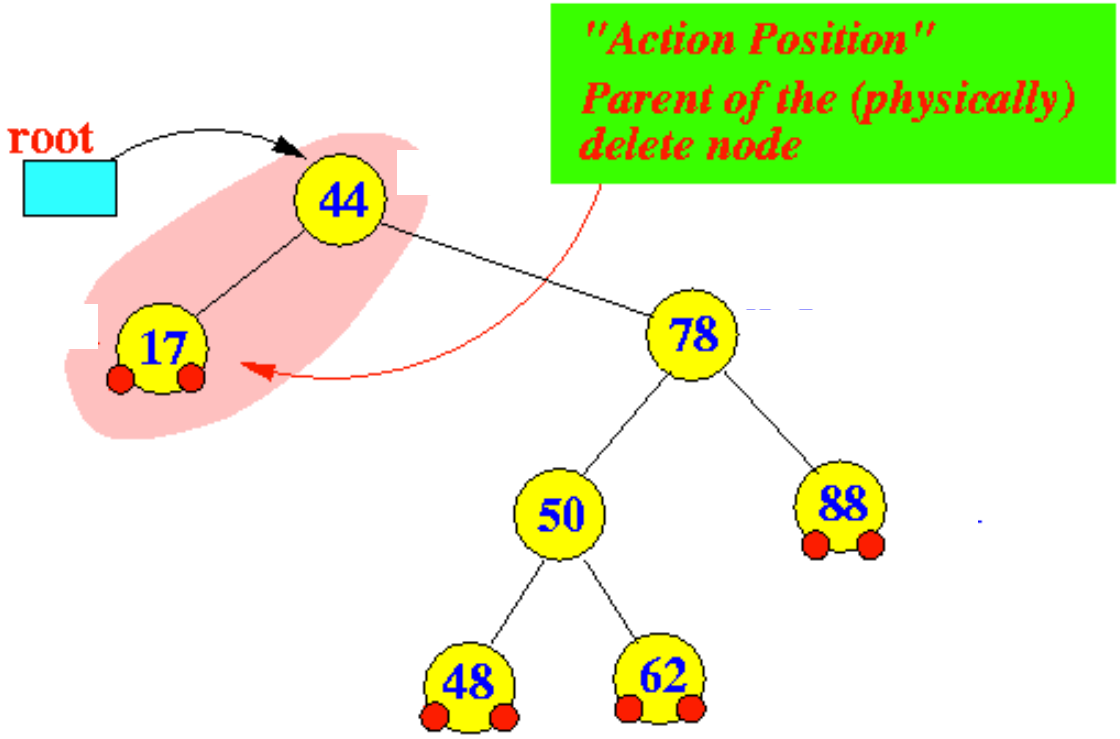
AVL Tree Deletion: Example

- Deleting a node from an ALV tree can cause imbalance
  - Consider deleting node 32



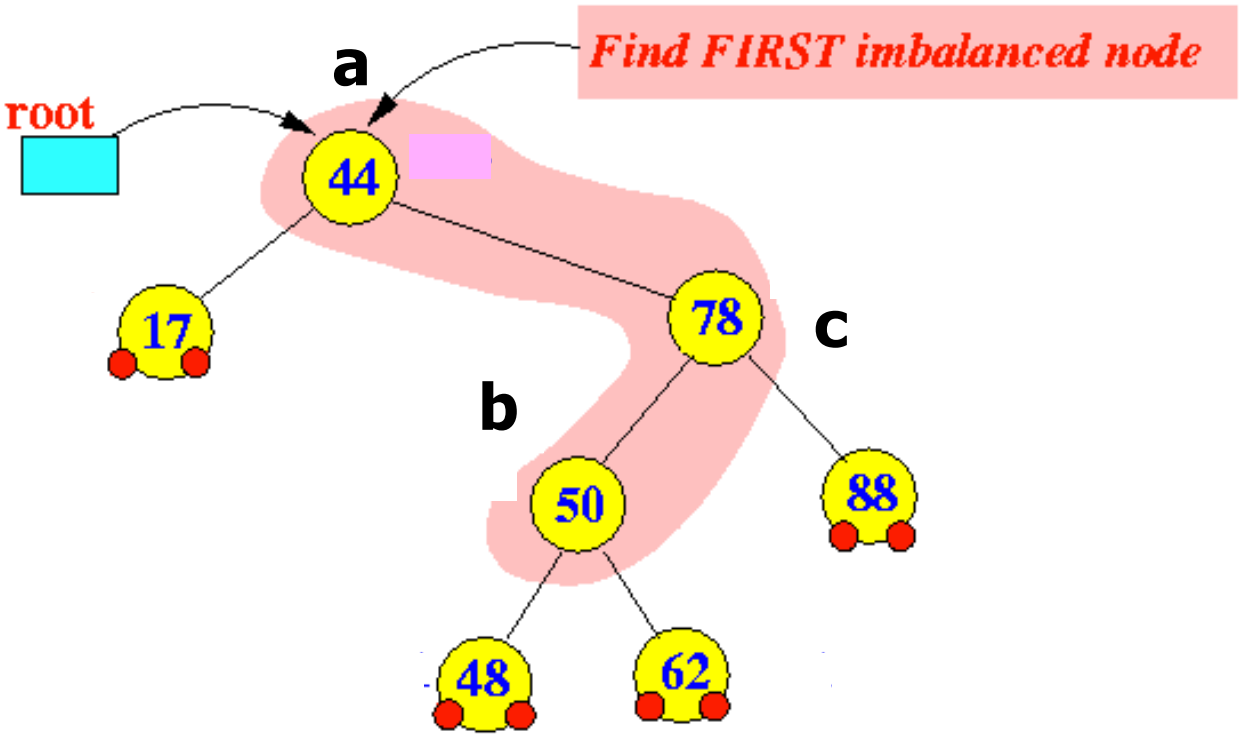
AVL Tree Deletion: Example

- The balance factor changes at only nodes between the root and the parent node of the physically deleted node
  - Starting at the action position find the first imbalanced node



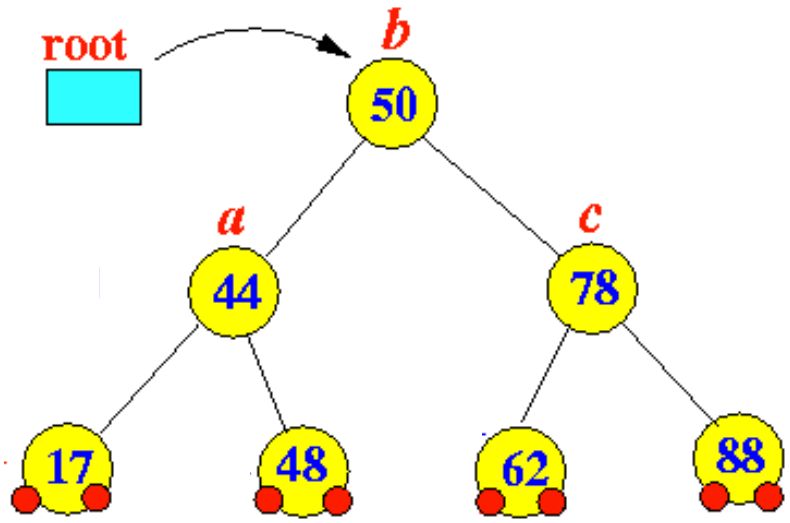
AVL Tree Deletion: Example

- Perform rotation using shaded nodes
  - Node a is the first imbalanced node from the action position
  - Node c is the child node of node a that has the higher height
  - Node b is the child node of node b that has the higher height



AVL Tree Deletion: Example

- The tree after LR rotation



AVL Tree Deletion: Multiple Imbalance

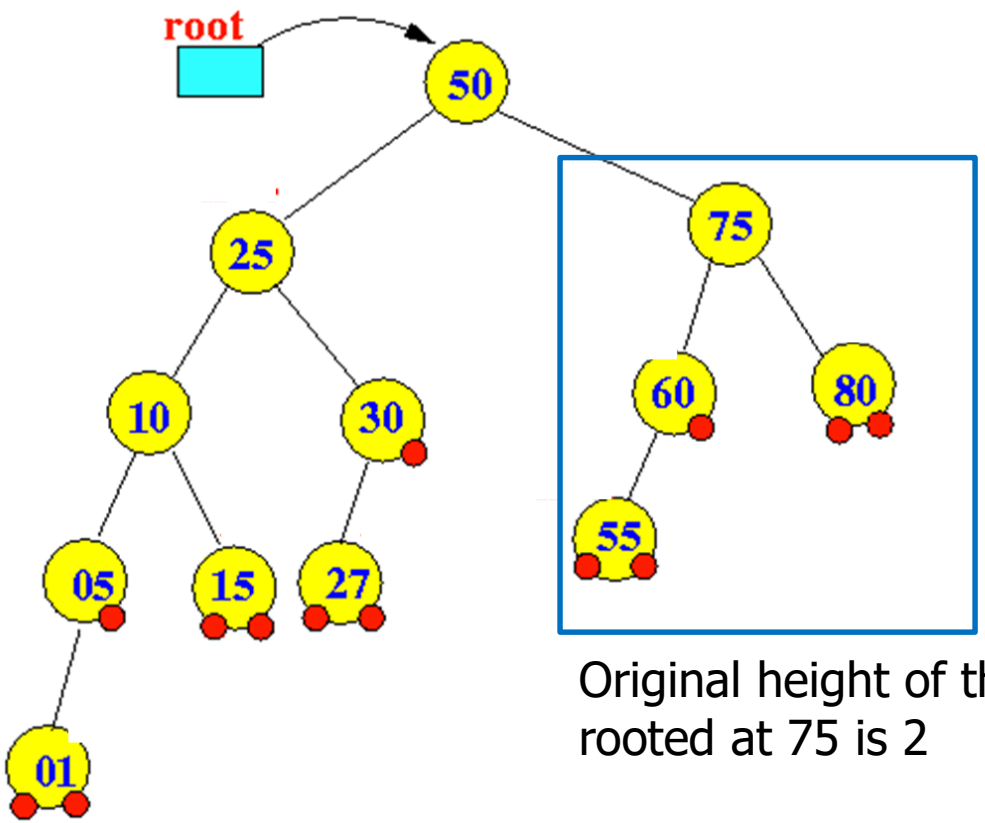
- The imbalance at the first imbalance node due to a deletion operation can be restored using rotation
  - Resulting subtree does not have the same height as the original subtree !!!
  - Nodes that are further up the tree may require re-balancing

- Deleting a node may cause more than one AVL imbalance !!!

- Unfortunately, delete may cause `O(h)` imbalances
  - Insertions will only cause one imbalance that must be fixed

AVL Tree Deletion: Example

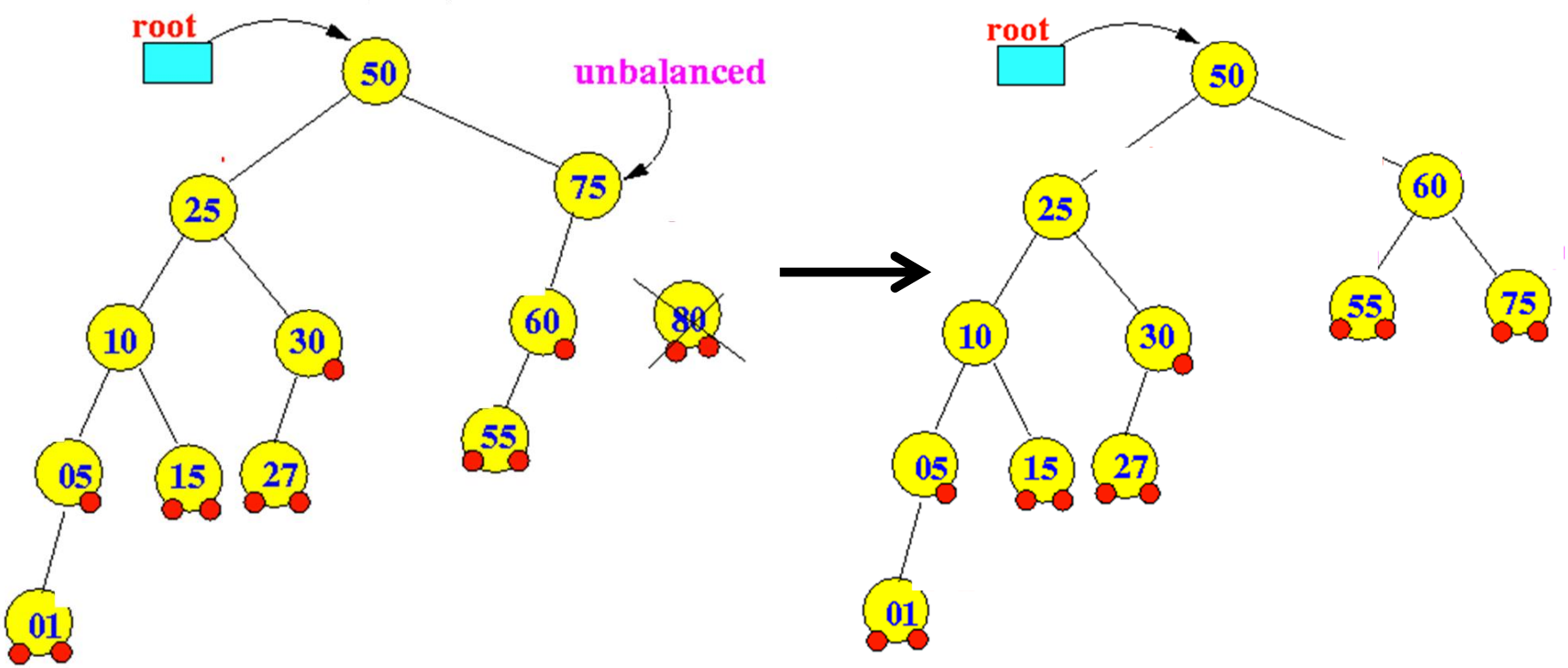
- Consider the following AVL tree



Original height of the subtree rooted at 75 is 2

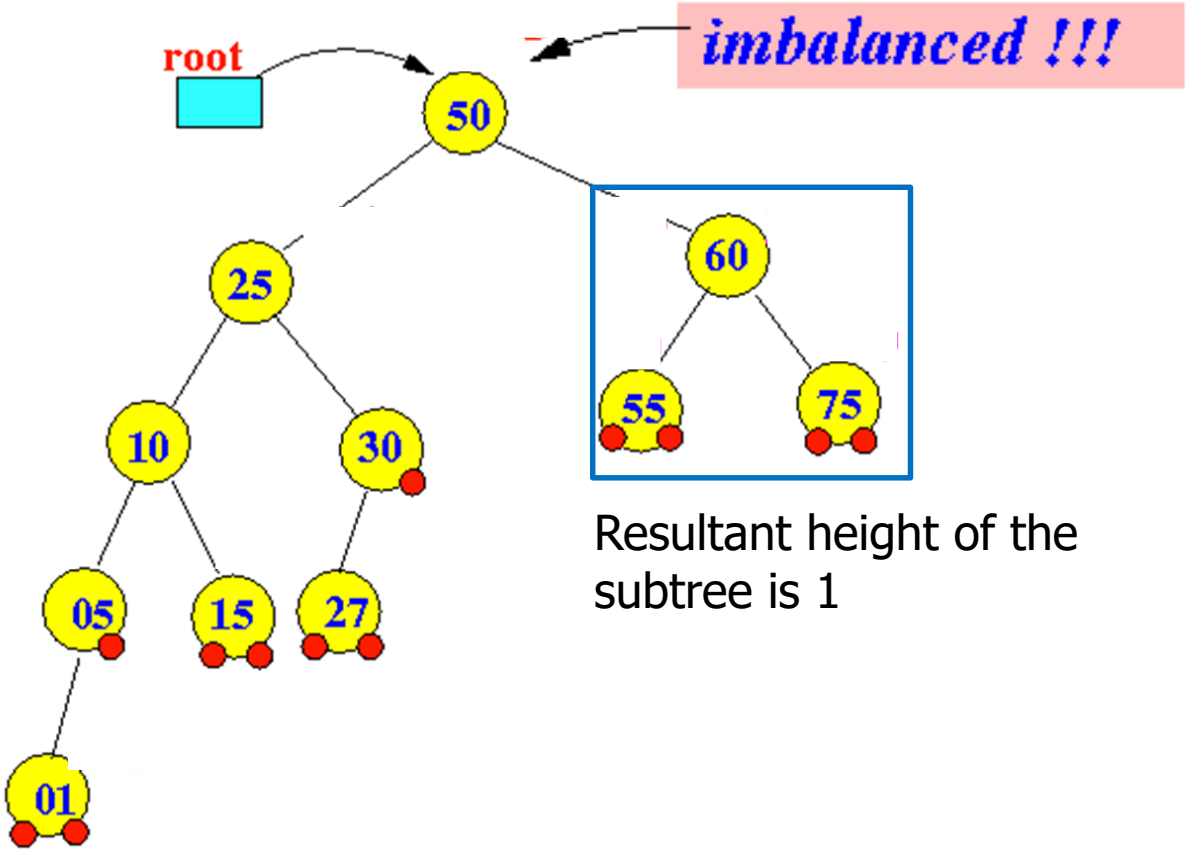
AVL Tree Deletion: Example

- Node with value 80 is deleted
  - The imbalance is on left-left subtree
  - Imbalance can be fixed using RR rotation



AVL Tree Deletion: Example

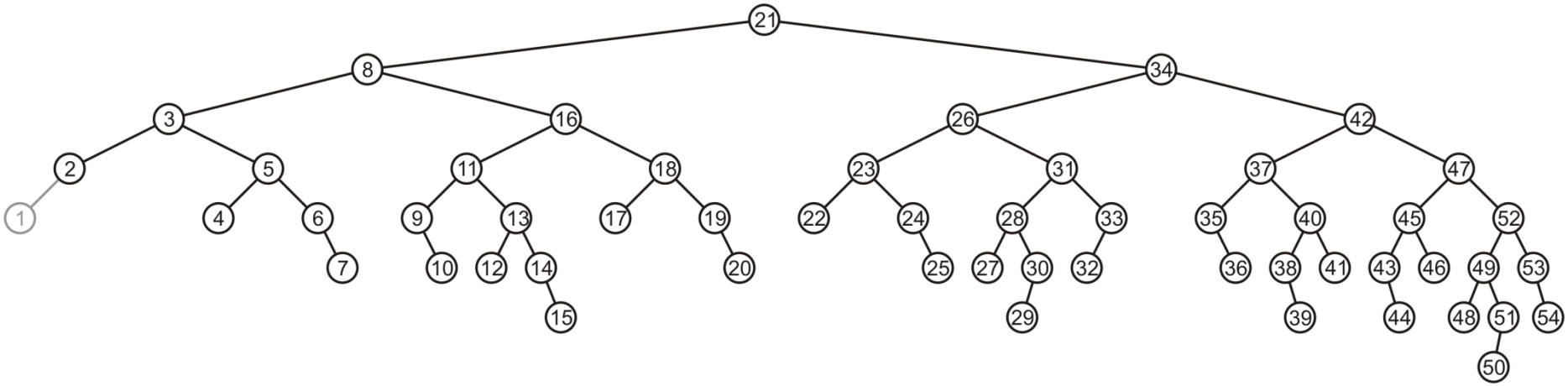
- Node 50 requires re-balancing
  - The imbalance is on left-left subtree
  - Imbalance can be fixed using RR rotation – Home work!!



Resultant height of the subtree is 1

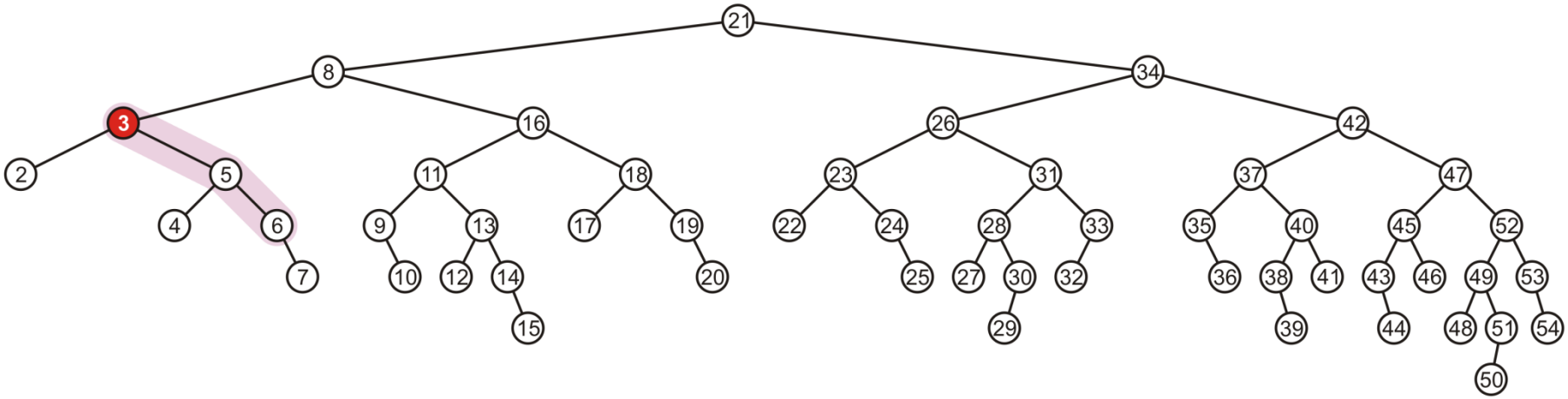
AVL Tree Deletion: Example

- Consider the following AVL tree
  - Suppose node with value 1 is deleted



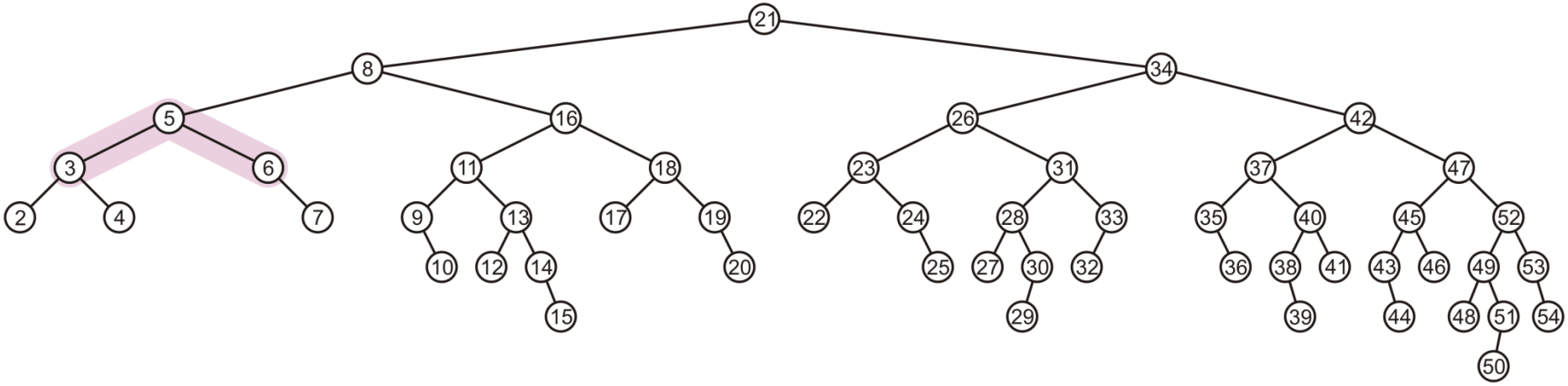
AVL Tree Deletion: Example

- While its previous parent, 2, is not unbalanced, its grandparent 3 is
  - The imbalance is in the right-right subtree



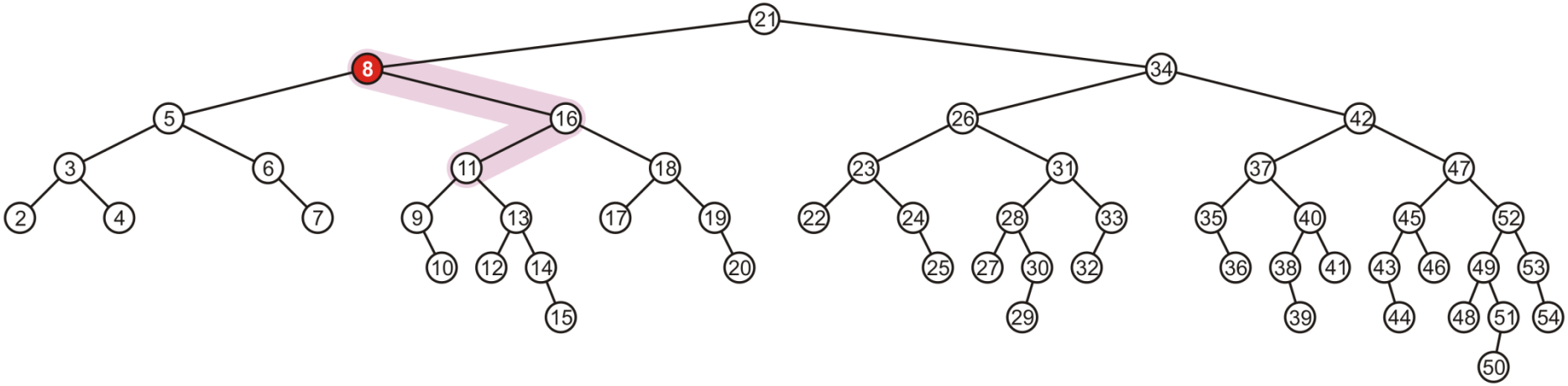
AVL Tree Deletion: Example

- While its previous parent, 2, is not unbalanced, its grandparent 3 is
  - The imbalance is in the right-right subtree
  - Imbalance can be fixed using LL rotation



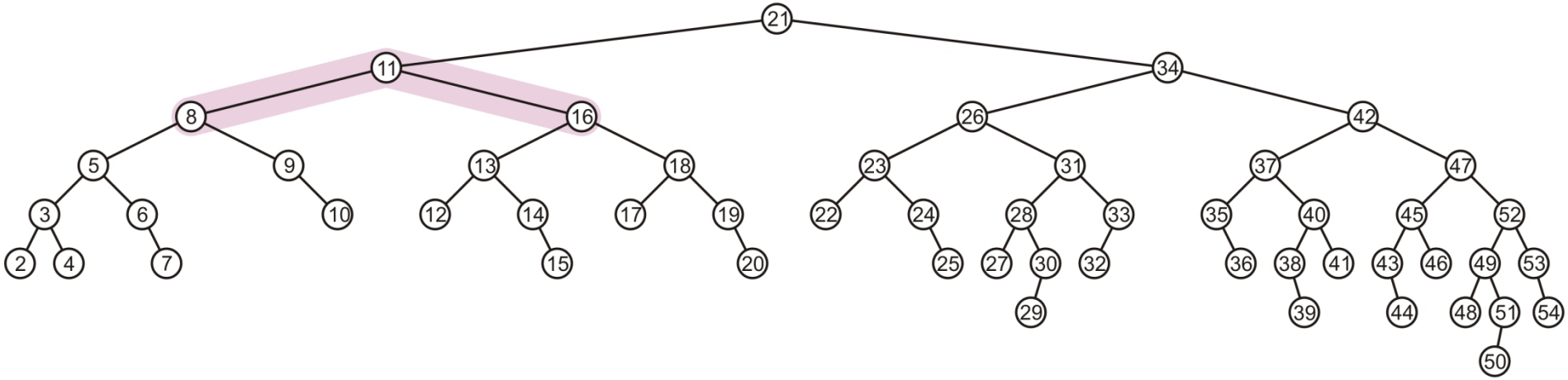
AVL Tree Deletion: Example

- The subtrees of node 5 is now balanced
- Recursing to the root, however, 8 is also unbalanced
  - The imbalance is in right-left subtree



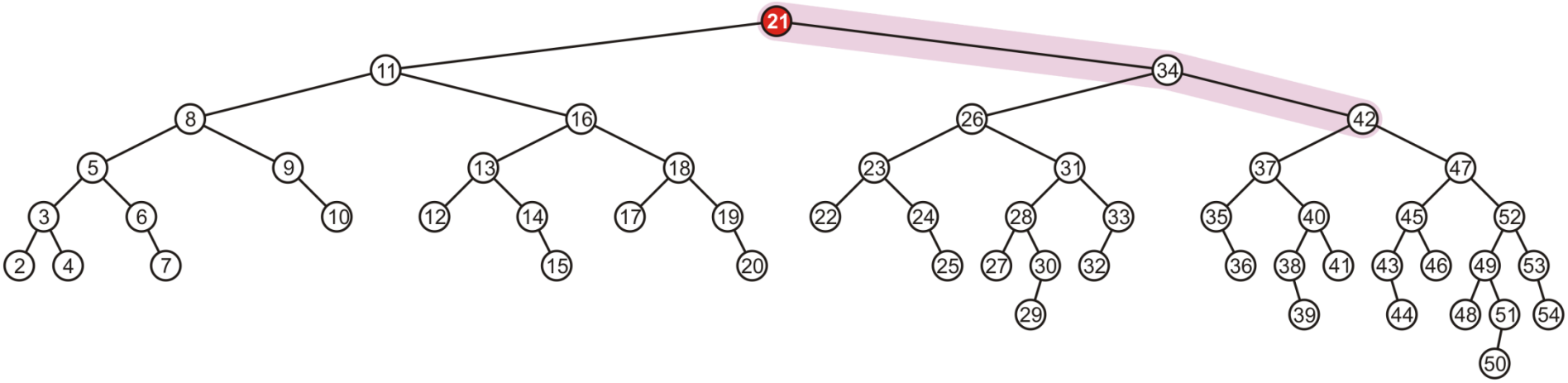
AVL Tree Deletion: Example

- The node with value 8 is unbalanced
  - The imbalance is in right-left subtree
  - LR rotation can fix imbalance



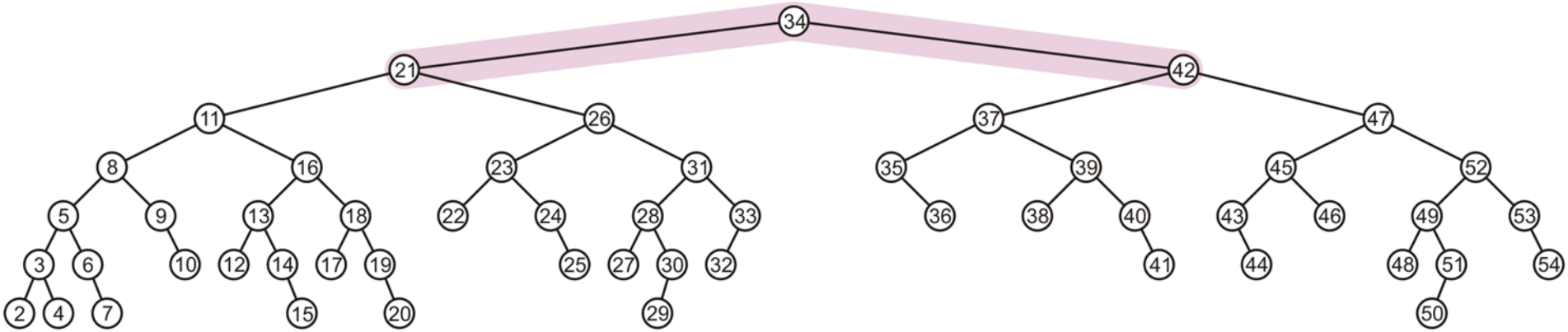
AVL Tree Deletion: Example

- Root 21 is still imbalanced
  - The imbalance is in right-right subtree



AVL Tree Deletion: Example

- Root 21 is still imbalanced
  - The imbalance is in right-right subtree
  - LL rotation can fix the imbalance



Any Question So Far?